

Trabajo Práctico — Parte 1: Síntesis lógica y análisis de timing con LibreLane (sky130)

Este trabajo práctico tiene dos partes:

- **Parte 1 (este documento):** síntesis lógica y análisis estático de timing pre-layout.
- **Parte 2 (se publicará más adelante):** implementación física completa con place-and-route: floorplan, placement, síntesis del árbol de clock (CTS), routing y signoff. Se va a trabajar sobre los mismos diseños, así que lo que aprendas acá sobre los reportes, las bibliotecas y las restricciones lo vas a seguir usando.

Objetivos

Al terminar este trabajo práctico deberías poder:

1. Correr un flujo de **síntesis lógica y análisis estático de timing (STA)** con herramientas open source (Yosys, OpenSTA) sobre un PDK real (SkyWater sky130).
2. Relacionar el **RTL** con el **netlist** de compuertas que genera la síntesis.
3. Leer e interpretar los reportes de **QoR** (*Quality of Results*): área, celdas, timing (setup/hold) y potencia.
4. Entender cómo influyen las **restricciones de timing** (período de clock, retardos de entrada/salida) y las **esquinas PVT** en los resultados.
5. Evaluar el impacto de elegir una **biblioteca de celdas estándar** distinta sobre el QoR de un diseño grande (un CPU RISC-V con memorias SRAM).

Antes de empezar

- Seguí la **sección 1 del README.md** (instalación de Docker, clonado con submódulos, contenedor de LibreLane y `make pdk`). Hacelo con tiempo: la primera vez se descargan ~15 GB.
- Leé las **secciones 2, 3 y 4 del README.md**. Ahí está explicado qué hace cada comando, dónde quedan los resultados, qué contiene cada reporte y dónde están los archivos de las bibliotecas de celdas (`.lib`, `.lef`). Este TP supone que ya lo leíste.
- Todos los comandos se corren desde la raíz del proyecto.

Cosas a tener en cuenta durante todo el TP

- Cada corrida crea un directorio nuevo en `designs/<diseño>/runs/(<variante>_<fecha>_<hora>)`. Anotá qué corrida corresponde a cada experimento.
- `runs/compare.md` se **sobrescribe** cada vez que corrés `make compare` o `make sta-all`. Si lo vas a usar en el informe, copialo con otro nombre.

- Si modificás un archivo de configuración o RTL para un experimento, **volvelo a su valor original** antes de pasar al siguiente (o usá `git diff` / `git checkout -- <archivo>` para controlarlo).
- Todos los resultados de timing son **pre-layout** (sin place-and-route): los clocks son ideales y los cables no tienen resistencia ni capacidad. Tenelo presente al sacar conclusiones.

Entrega

Un **informe en PDF** que contenga, para cada parte:

- Las **tablas** pedidas, con los valores obtenidos y el nombre de la corrida de la que salen.
- Las **respuestas** a cada pregunta, justificadas con datos de los reportes (citá el archivo y el valor). Una respuesta sin justificación con datos no se considera completa.
- Los **fragmentos relevantes** de reportes o del netlist (no pegues reportes completos).

Las preguntas están numeradas (**P1**, **P2**, ...): usá esa numeración en el informe.

Uso de herramientas de IA y defensa del trabajo

- **Está permitido usar chats o agentes de IA** (ChatGPT, Claude, Copilot, etc.) para hacer este trabajo: para entender conceptos, interpretar reportes, escribir comandos o revisar el informe.
- En la **evaluación final el trabajo deberá ser defendido** oralmente. Vas a tener que explicar cada respuesta y cada número de tu informe, de dónde sale y qué significa, sin ayuda. Usá la IA para aprender, no para entregar algo que no entendés.

Ejercicio 1: Un contador de 8 bits

El diseño está en `designs/counter/`. Es un contador con enable y reset asincrónico activo en bajo.

1.1 Del RTL al netlist

Qué hacer:

1. Leé `designs/counter/src/counter.v`, `config.json` y `constraints.sdc`.
2. Corré la síntesis:
`make synth`
3. Abrí el netlist generado (`<run>/final/nl/counter.nl.v`) y el reporte `<run>/reports/gates.md`.

Qué se debe lograr: entender qué celdas usó la herramienta para implementar el RTL y por qué.

- **P1.** ¿Cuántos flip-flops tiene el netlist y de qué celda son? ¿Coincide con lo que esperabas a partir del RTL? Usando la sección 4.5 del `README.md` y la documentación de sky130 (o deduciéndolo del nombre y de los pines de la celda), explicá qué significa cada letra de `dfirtp` y por qué la herramienta eligió esa celda y no otra.
- **P2.** Elegí el bit menos significativo del contador (`count[0]`) y seguilo en el netlist: dibujá el circuito (flip-flop + lógica combinacional) que lo implementa y verificá que es funcionalmente correcto (incluyendo el enable).
- **P3.** Según `gates.md`, ¿qué porcentaje del área es secuencial y qué porcentaje combinacional? ¿Qué celda es la más grande por instancia?

1.2 Análisis de timing

Qué hacer:

```
make sta
```

Abrí `<run>/reports/timing.md`.

Qué se debe lograr: saber leer un camino crítico línea por línea y entender de dónde sale cada número.

- **P4.** Completá esta tabla con los resultados de las tres esquinas:

Esquina	Setup WS (ns)	Hold WS (ns)
<code>nom_tt_025C_1v80</code>		
<code>nom_ss_100C_1v60</code>		
<code>nom_ff_n40C_1v95</code>		

¿En qué esquina es peor el setup y en cuál el hold? Explicá por qué, en términos de proceso, tensión y temperatura.

- **P5.** Para el **camino crítico de setup**:
 - Indicá el punto de inicio y el de fin, y de qué tipo es cada uno (puerto de entrada, flip-flop, puerto de salida).
 - El *input external delay* y la *clock uncertainty* son restricciones: indicá en qué línea del `constraints.sdc` se define cada uno y verificá que el valor del reporte coincide.
 - El *library setup time* no está en el SDC: es una propiedad del flip-flop, que la herramienta saca del `.lib` de la biblioteca en esa esquina. Para este camino tomó **`t_setup = 0.253 ns`**.

- Calculá el *data required time* (el presupuesto de tiempo: período, incertidumbre y setup) y, con el *data arrival time* del reporte, el *slack*. Mostrá la cuenta.
- (*Opcional*) Sumá los retardos de cada celda del camino y verificá que obtenés el *data arrival time* del reporte.
- **P6.** Para el **camino crítico de hold**: ¿entre qué elementos es? ¿Por qué un camino tan corto es el peor caso de hold? ¿Qué pasaría si el slack de hold fuese negativo, y por qué **no** se arregla bajando la frecuencia?

1.3 Escalado con el ancho

Qué hacer: cambiá `WIDTH` a 32 en `src/counter.v` y corré `make sta` (con el período en 10 ns).

Qué se debe lograr: ver cómo escalan área y retardo con el tamaño del diseño.

- **P7.** Compará 8 bits vs 32 bits: cantidad de flip-flops, área secuencial, área combinacional, retardo del camino crítico (*data arrival time* menos el *input external delay*, o el equivalente según el tipo de camino). ¿El área crece linealmente con el ancho? ¿Y el retardo? ¿Por qué?
- **P8.** Describí el camino crítico de la versión de 32 bits. ¿Qué parte del contador lo forma?

Al terminar, **volvé `WIDTH` a 8**.

Ejercicio 2: Un CPU RISC-V (Ibex) con memorias SRAM

El diseño está en `designs/ibex/`; la sección 3 del `README.md` describe cómo está armado. Cada corrida tarda unos 2 minutos.

2.1 Entender la configuración

Qué hacer: leé `designs/ibex/config.json`, `constraints.sdc` y `src/prim_ram_1p.sv`.

Qué se debe lograr: entender qué se está sintetizando antes de mirar números.

- **P9.** La caché de instrucciones tiene 2 vías, cada una con una RAM de tags de 22 bits $\times$ 256 y una RAM de datos de 64 bits $\times$ 256. La macro SRAM disponible es de 32 bits $\times$ 256. ¿Cuántos bits de memoria necesita la caché y cuántos bits proveen las 6 macros? ¿Qué porcentaje se desperdicia y dónde?
- **P10.** En la sección `MACROS` de `config.json`, ¿qué vista de la macro usa la síntesis y cuál usa el STA? ¿Por qué la síntesis la trata como una caja negra?

- **P11.** Mirá la entrada "lib" de la macro (y la sección 4.3 del README.md). ¿Para qué esquina PVT está caracterizada la SRAM? ¿Qué implica eso para la precisión del análisis en las esquinas **ss** y **ff**?

2.2 Síntesis con la biblioteca de alta densidad (hd)

Qué hacer:

```
make sta DESIGN=ibex
```

Qué se debe lograr: hacer una lectura crítica del QoR de un diseño grande.

- **P12.** Según qor.md: ¿hubo errores o warnings de lint, latches inferidos o celdas sin mapear? ¿Por qué es importante verificar cada uno de esos ítems antes de mirar área y timing?
- **P13.** Según area.md y gates.md: ¿qué fracción del área total son las macros? Dentro de las celdas estándar, ¿cuántos flip-flops hay y de qué tipos? El banco de registros de RV32 tiene 31 registros de 32 bits útiles: ¿qué fracción de los flip-flops representaría? ¿Qué celdas combinatoriales dominan y con qué estructura del CPU las relacionás?
- **P14.** En timing.md, analizá el camino crítico de setup:
 - ¿Dónde empieza y dónde termina? ¿En qué flanco del clock arranca y en qué instante (ns)? ¿Cuánto tiempo queda, entonces, para la lógica del camino? Hací el presupuesto completo: período, flanco de inicio, retardo de salida, incertidumbre.
 - (Opcional) Sumá los retardos de las celdas del camino y verificá el *data arrival time* del reporte.
 - Buscá en el camino nets con **fanout** grande y mirá su **slew**. ¿Qué efecto tienen en el retardo de la celda que las maneja?
- **P15.** qor.md reporta miles de violaciones de *max slew* y *max cap*. Relacionalo con lo que viste en P14. ¿Por qué es esperable en este punto del flujo, y qué etapa posterior lo resuelve?

2.3 Comparación de bibliotecas: hd vs hs

sky130 tiene varias bibliotecas de celdas estándar. En este TP comparamos:

- sky130_fd_sc_hd (**high density**): celdas chicas, bajo consumo.
- sky130_fd_sc_hs (**high speed**): celdas más grandes y rápidas.

Lo único que cambia entre las dos corridas es la biblioteca (`designs/ibex/variants/*.json`); el RTL, las restricciones y la frecuencia son los mismos.

Qué hacer:

```
make sta-all DESIGN=ibex
```

Esto corre ambas variantes y genera `designs/ibex/runs/compare.md`.

Qué se debe lograr: cuantificar el impacto de la biblioteca en cada métrica de QoR y explicar el porqué de cada diferencia.

- **P16.** Armá esta tabla a partir de `compare.md`, agregando la columna de relación:

Métrica	hd	hs	hs / hd
Celdas			
Área de celdas estándar (μm^2)			
Área secuencial (μm^2)			
Área total (μm^2)			
Setup WS ss (ns)			
Setup WS tt (ns)			
Setup WS ff (ns)			
Hold WS (ns)			
Violaciones max slew / max cap			
Potencia tt (mW)			
Potencia ss (mW)			
Potencia ff (mW)			

- **P17. Área.**

- Usando `gates.md` de cada corrida, calculá el **área por instancia** de 3 celdas presentes en ambas bibliotecas (por ejemplo `dfrtp_2`, `mux2_1` y alguna compuerta simple). ¿En qué proporción son más grandes las celdas **hs**? Buscá la **altura de celda** de cada biblioteca en los `.lef` (sección 4.4 del `README.md`) y relacioná.
- El área de celdas estándar cambia bastante, pero el área total cambia poco. ¿Por qué? ¿Qué conclusión sacás sobre dónde conviene optimizar área en este diseño?

- **P18. Mapeo.** Compará la lista de celdas de `gates.md` en ambas corridas. ¿Usó la herramienta las mismas celdas en las mismas cantidades? Mostrá al menos dos diferencias concretas y proponé una explicación. ¿Cambió la cantidad de flip-flops? ¿Por qué tiene sentido?

- **P19. Timing.**

- Calculá el **retardo de la lógica del camino crítico** (no el slack) para cada biblioteca en **ss**. ¿Cuánto más rápida es **hs**?
- Compará los caminos críticos de ambas corridas: ¿empiezan y terminan en el mismo lugar? ¿Pasan por la misma cantidad de celdas?
- Comparando el setup slack de cada esquina, la mejora de **hs** sobre **hd**, ¿es igual en las tres? ¿En cuál gana más? Proponé una explicación.
- ¿Qué pasa con el slack de **hold** al usar celdas más rápidas? ¿Por qué?

- **P20. Potencia.**

- Para cada biblioteca y cada esquina, armá una tabla con la potencia **interna, de conmutación y de fuga (leakage)** del total, sacada de `power.md`.

- Calculá el leakage de las **celdas estándar** solamente (filas *Sequential* + *Combinational*). ¿Cuántas veces más leakage tiene **hs** que **hd** en cada esquina? ¿Por qué hay que excluir la fila *Macro* para que la comparación tenga sentido? Explicá por qué la temperatura, la tensión y el tipo de transistor afectan tanto a la fuga.
- En **hd** la esquina de mayor potencia es **ff**, pero en **hs** el orden entre **tt** y **ss** cambia respecto de **hd**. ¿Por qué?
- ¿La potencia de las macros cambia entre bibliotecas? ¿Por qué?

Criterios de evaluación

Se valora especialmente que las respuestas estén **justificadas con números de los reportes** y que las explicaciones relacionen los resultados con los conceptos de la materia (retardo de compuertas, fanout, setup/hold, esquinas PVT, potencia dinámica y estática).